

Computer Organization and Architecture

Chapter 5 Internal and External Memory



Characteristics

- Location
 - Capacity
 - Unit of transfer
 - Access method
 - Performance
 - Physical type
 - Physical characteristics
 - Organisation
- 
- The background of the slide is a solid blue color. In the bottom right corner, there are several faint, concentric circles that resemble ripples in water, creating a decorative effect.

Location

- Internal - Primary Memory
 - e.g. processor registers, cache, main memory (RAM)
- External - Secondary Memory
 - optical disks, magnetic disks, tapes

Capacity

- Word size
 - The natural unit of organisation
- Number of words
 - or Bytes

Unit of Transfer

➤ Internal

- Usually governed by data bus width

➤ External

- Usually a block which is much larger than a word

➤ Addressable unit

- Smallest location which can be uniquely addressed
- Word internally

Access Methods

➤ Sequential

- Start at the beginning and read through in order
- Access time depends on location of data and previous location
- e.g. tape

➤ Direct

- Individual blocks have unique address
- Access is by jumping to vicinity plus sequential search
- Access time depends on location and previous location
- e.g. disk

Access Methods

➤ Random

- Individual addresses identify locations exactly
- Access time is independent of location or previous access
- e.g. RAM

➤ Associative

- Data is located by a comparison with contents of a portion of the store
- Access time is independent of location or previous access
- e.g. cache

Performance

➤ Access time

- Time between presenting the address and getting the valid data stored for use (perform read and write operations)

➤ Memory Cycle time

- Time may be required for the memory to “recover” before next access
- Cycle time is access + recovery (access time + time required before a 2nd access can commence)

Performance

➤ Transfer Rate

- Rate at which data can be moved (in and out of memory)
- RAM: $1 / (\text{cycle time})$
- Non-RAM: $T_N = T_A + N/R$

Where:

T_N = Average time to read or write N bits

T_A = Average access time

N = Number of bits

R = Transfer rate, in bits per second (bps)

Sample Problem

- Given: You might have transferred 25 GB in 2 minutes. Compute the transfer rate.

Solution: time = 2 minutes = 120 seconds

$$\text{Transfer Rate} = 25 / 120 = 0.208$$

$$\text{Transfer Rate} = 0.208 \text{ GB/s}$$

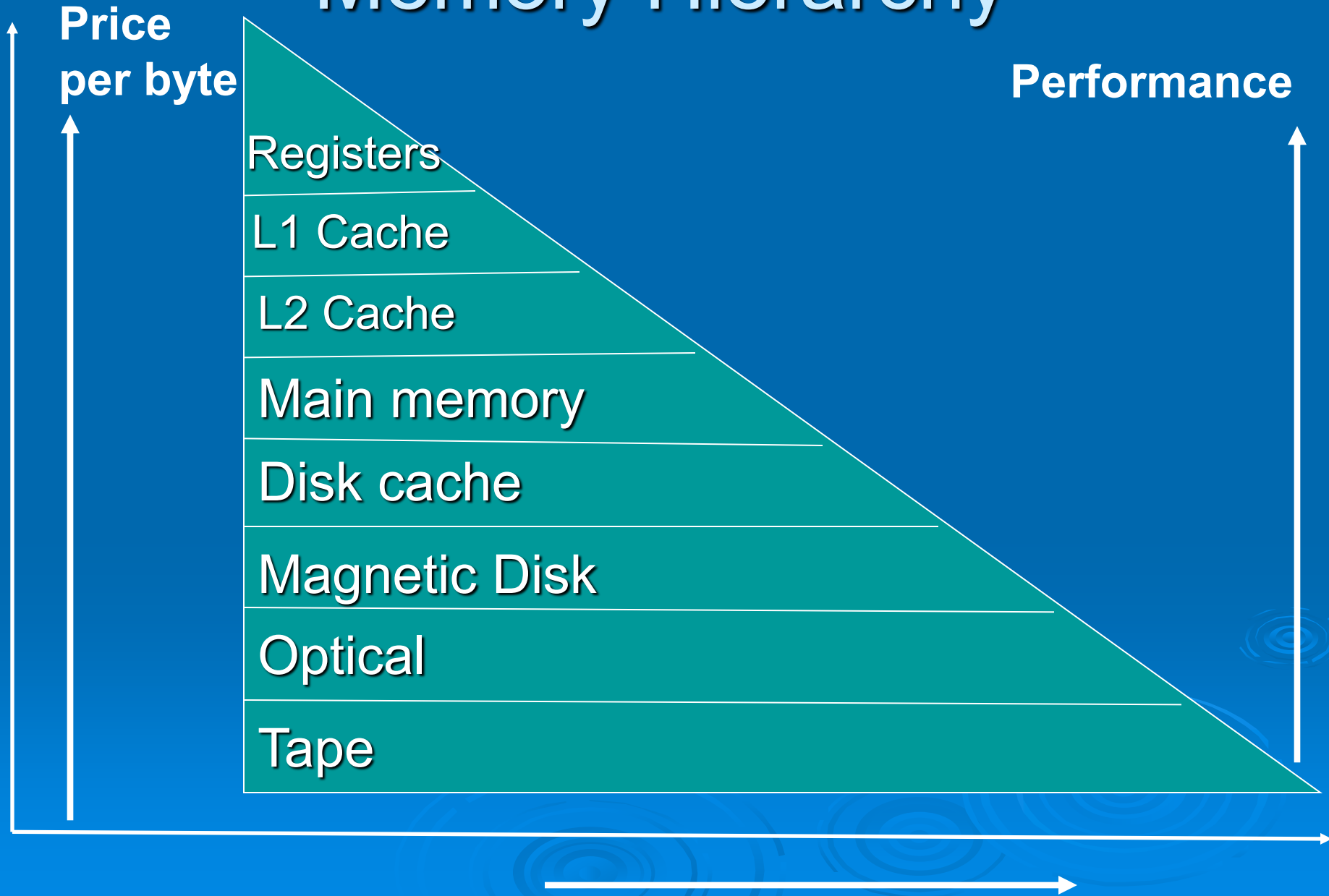
Memory Hierarchy

- Registers
 - In CPU
- Internal or Main memory
 - May include one or more levels of cache
 - “RAM”
- External memory
 - Backing store

Memory Hierarchy Concept

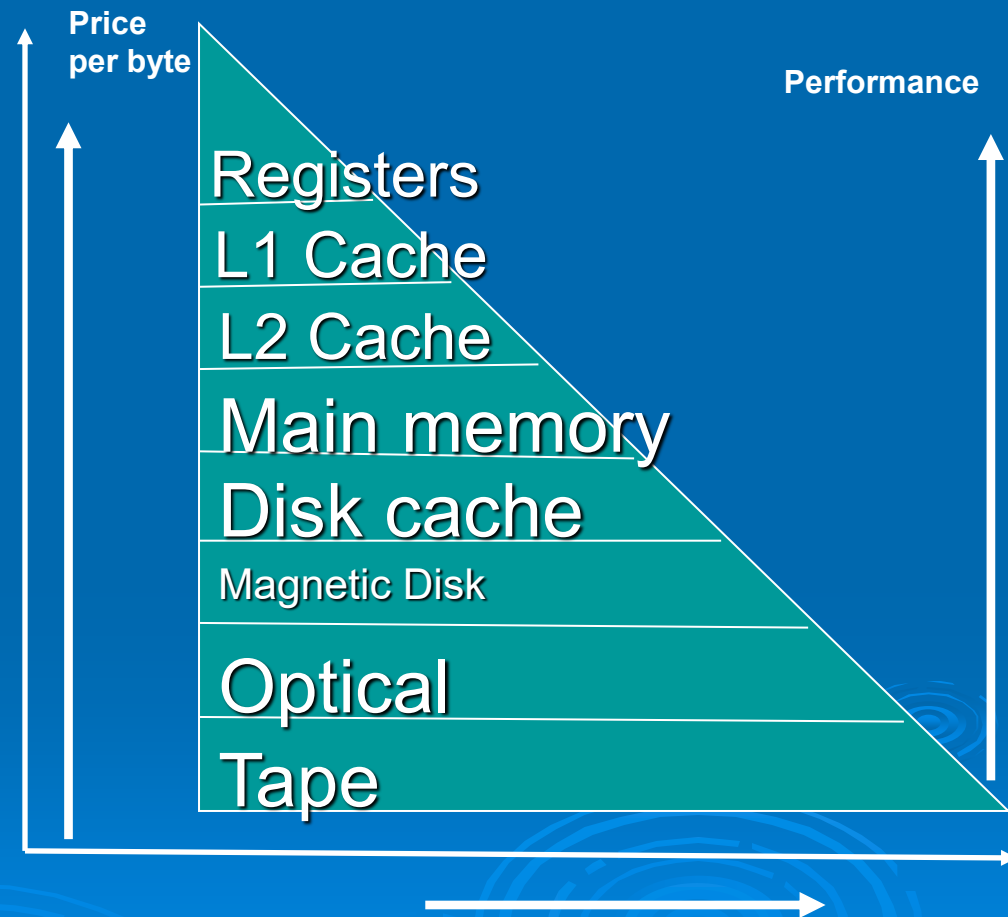
- Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time.
- Based on a program behavior - locality of references
 - Temporal Locality
 - Spatial Locality

Memory Hierarchy



Memory Hierarchy

- Faster access time, greater cost per bit
- Greater capacity, smaller cost per bit
- Greater capacity, slower access time
- Decreasing cost per bit
- Increasing capacity
- Increasing access time
- Decreasing frequency of access of the memory by the processor



Memory Hierarchy Constraints

- Design constraints on computer's memory can be summed up by three question
 - How much?
 - If the Capacity is there the application will use
 - How fast?
 - To achieve greatest performance, the memory must be able to keep up with the processor
 - Processor should not pause waiting for instructions or operands
 - How expensive?
 - Must be reasonable in relationship to other components

Effectiveness of Memory Hierarchy

$$T_A = h * M_1 + (1-h)(M_1 + M_2)$$

where :

h = hit rate

M_1 = access time of 1st level memory

M_2 = access time of 2nd level memory

$$\text{HIT RATIO} = \frac{HIT}{HIT + MISS}$$

Hit and Miss

- Occurrence of Hit— in a level when the address that an operation references is found in that level of the memory hierarchy
- Otherwise, a Miss
- Access frequency to M_i is defines as

$$f_i = (1-h_1)(1-h_2)....(1-h_i)$$

Effective Access Time (T_{eff})

- The T_{eff} of a memory hierarchy is given by:

$$\begin{aligned} T_{eff} &= \sum_{i=1}^n f_i \cdot t_i \\ &= h_1 t_1 + (1 - h_1) h_2 t_2 + (1 - h_1)(1 - h_2) h_3 t_3 + \dots + \\ &\quad (1 - h_1)(1 - h_2) \dots (1 - h_{n-1}) t_n \end{aligned}$$

- Hierarchy Optimization - The total cost of a memory hierarchy is estimated as:

- c_i = cost in level i
- s_i = capacity in level i

$$C_{total} = \sum_{i=1}^n c_i \cdot s_i$$

Hit and Miss Rates

- Hit rate of a level— the percentage of references that reach the level that result in hits
- Miss rate of a level— the percentage of references that reach the level that result in misses

Hit rate + Miss Rate at a level

➤ = 100%

- Neither the hit rate nor the miss rate count the references that are handled by higher levels in the hierarchy
- For example, requests that hit in the cache of our example memory hierarchy do not count in the hit rate or miss rate of the main memory

Average Access Times in a memory hierarchy

- First know the hit rate and access time (time to complete a request that hits) for each level in the memory hierarchy
- Then compute the average access time of the memory hierarchy

Average access time T_{av} at a level

- $T_{av} = (T_{hit} \times P_{hit}) + (T_{miss} \times P_{miss})$
- T_{hit} = The time taken to resolve requests that hit in the level,
- P_{hit} = The hit rate of the level (expressed as a probability)
- T_{miss} = The average access time of the levels below this one in the hierarchy, and
- P_{miss} = The miss rate of the level

Hit rate of the lowest Level

- Hit rate of the lowest level in the hierarchy
- = 100 percent (all requests that reach the bottom level are handled by the bottom level)
- Start at the bottom level and work up to compute the average access time of each level in the hierarchy

Physical Types

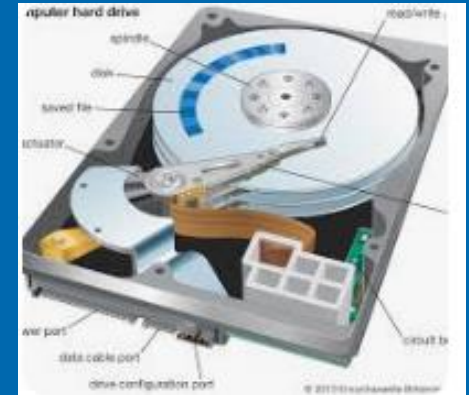
➤ Semiconductor

- RAM



➤ Magnetic

- Disk & Tape



➤ Optical

- CD & DVD



➤ Others

- Bubble
- Hologram



HOLOGRAPHIC MEMORY



Physical Characteristics

➤ Volatility

- Volatile memory – information decays naturally or is lost when electrical power is switched off
- Non volatile – information remains without deterioration until deliberately changed (magnetic surface)

➤ Erasable

- Non erasable memories cannot be altered except by destroying the storage unit e.d ROM

➤ Power consumption -

Semiconductor Memory

➤ RAM

- Misnamed as all semiconductor memory is random access
- Read/Write
- Volatile
- Temporary storage
- Static or dynamic

Dynamic RAM

- Bits stored as charge in capacitors
- Charges leak
- Need refreshing even when powered
- Simpler construction
- Smaller per bit
- Less expensive
- Need refresh circuits
- Slower
- Main memory

Static RAM

- Bits stored as on/off switches
- No charges to leak
- No refreshing needed when powered
- More complex construction
- Larger per bit
- More expensive
- Does not need refresh circuits
- Faster
- Cache

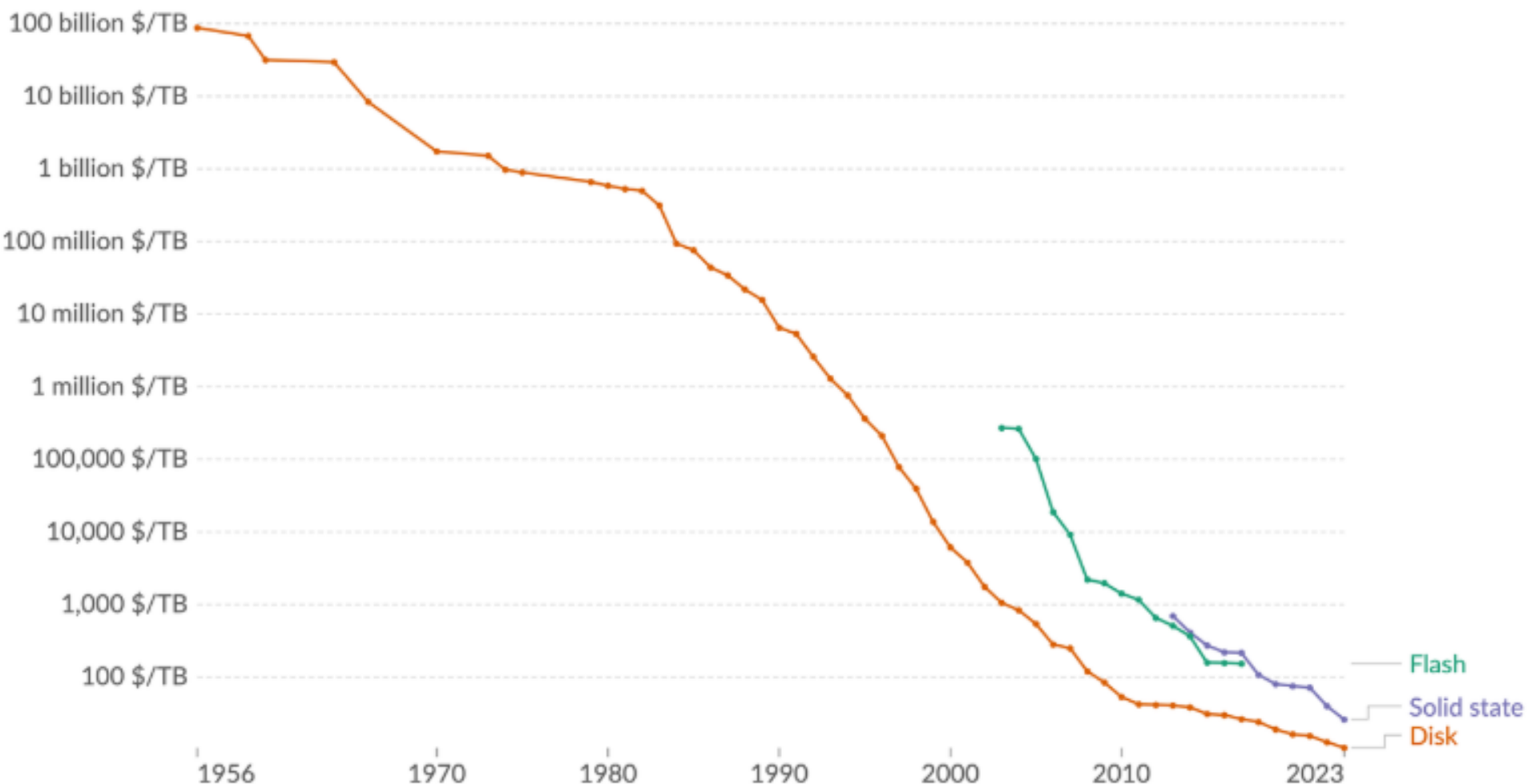
Read Only Memory (ROM)

- Permanent storage
- Microprogramming (see later)
- Library subroutines
- Systems programs (BIOS)
- Function tables

Types of ROM

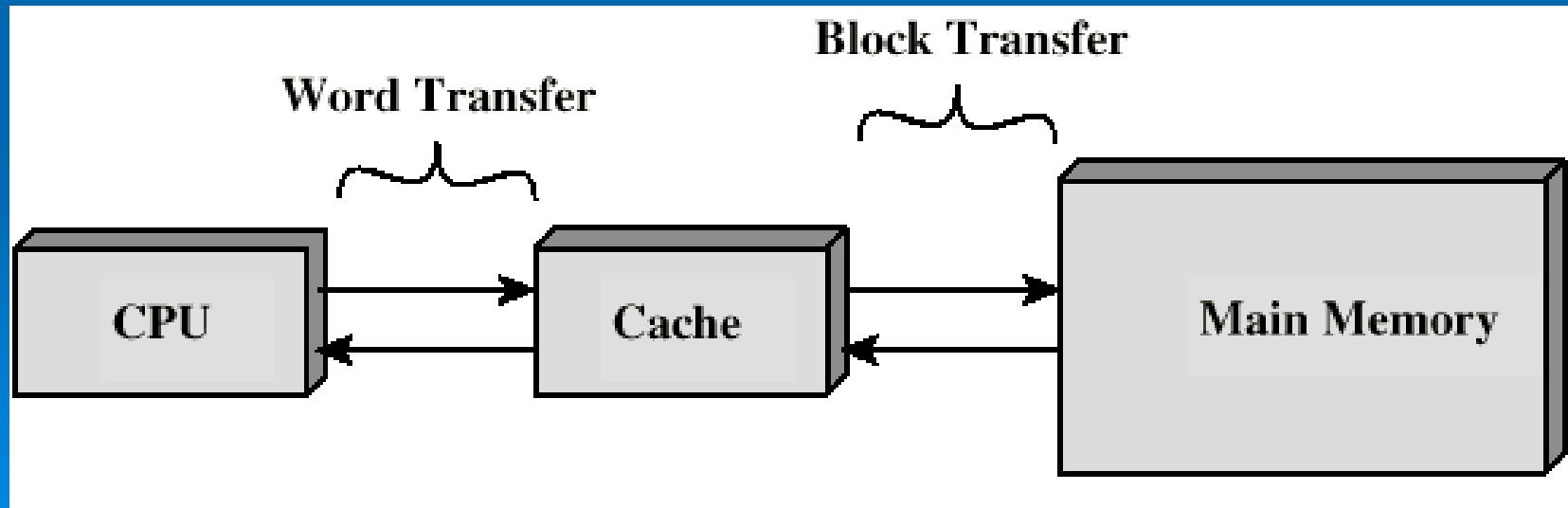
- Written during manufacture
 - Very expensive for small runs
- Programmable (once)
 - PROM
 - Needs special equipment to program
- Read “mostly”
 - Erasable Programmable (EPROM)
 - Erased by UV
 - Electrically Erasable (EEPROM)
 - Takes much longer to write than read

Historical Cost of Computer Memory and Storage



Cache

- Small amount of fast memory
- Sits between normal main memory and CPU
- May be located on CPU chip or module
- Holds the most recently accessed code or data



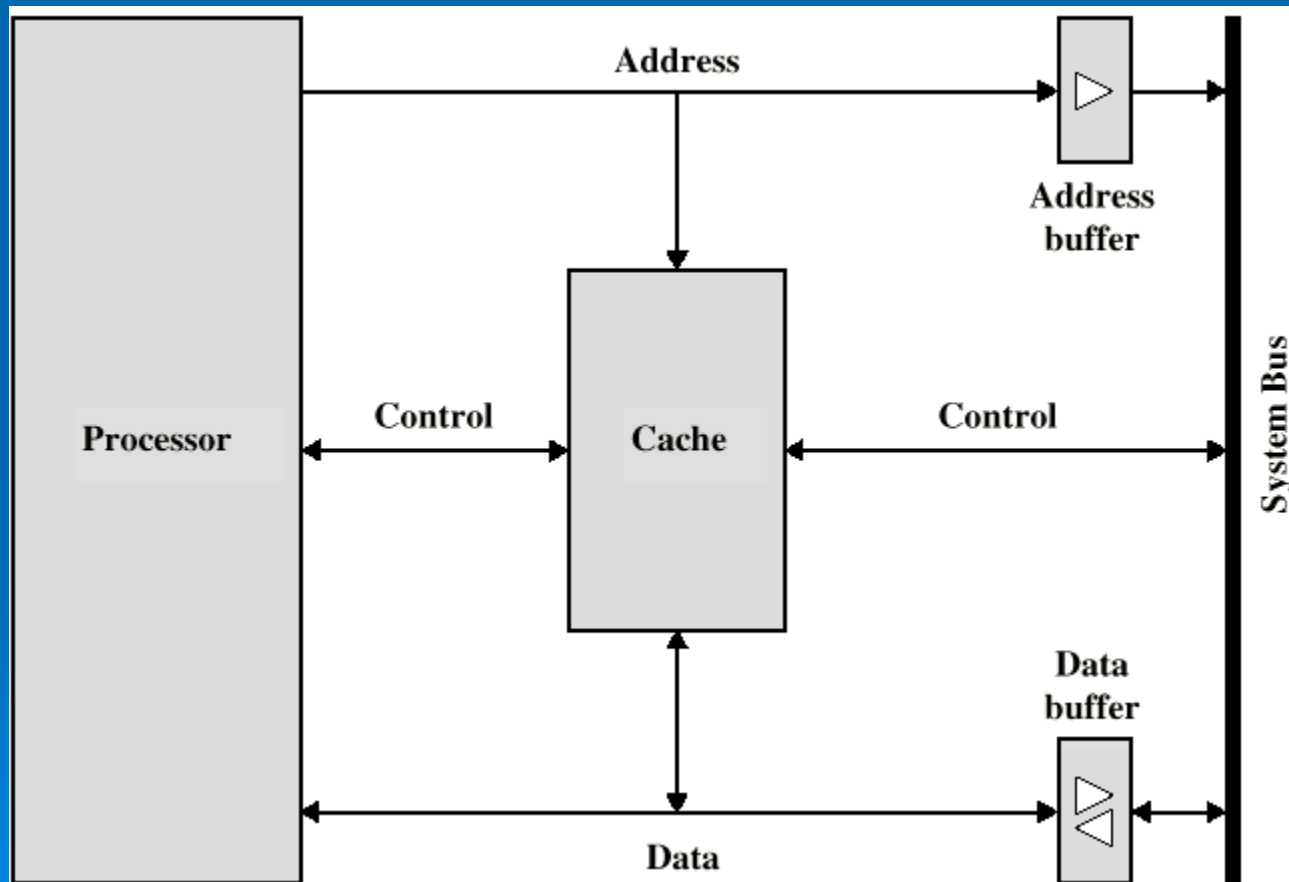
Cache Design

- Size
- Mapping Function
- Replacement Algorithm
- Write Policy
- Block Size
- Number of Caches

Cache operation - overview

- CPU requests contents of memory location
- Check cache for this data
- If present, get from cache (fast)
- If not present, read required block from main memory to cache
- Then deliver from cache to CPU
- Cache includes tags to identify which block of main memory is in each cache slot

Typical Cache Organization



➤ What Is A Cache Line?

- Data is hauled into the cache from memory in “chunks”.
- If you ask for 4 bytes of data, you’ll get the whole line (32/64/128 bytes)
- Locality of reference says you’ll need that data anyway

➤ How Is The Cache Laid Out?

■ **The cache is made up of a number of cache lines.**

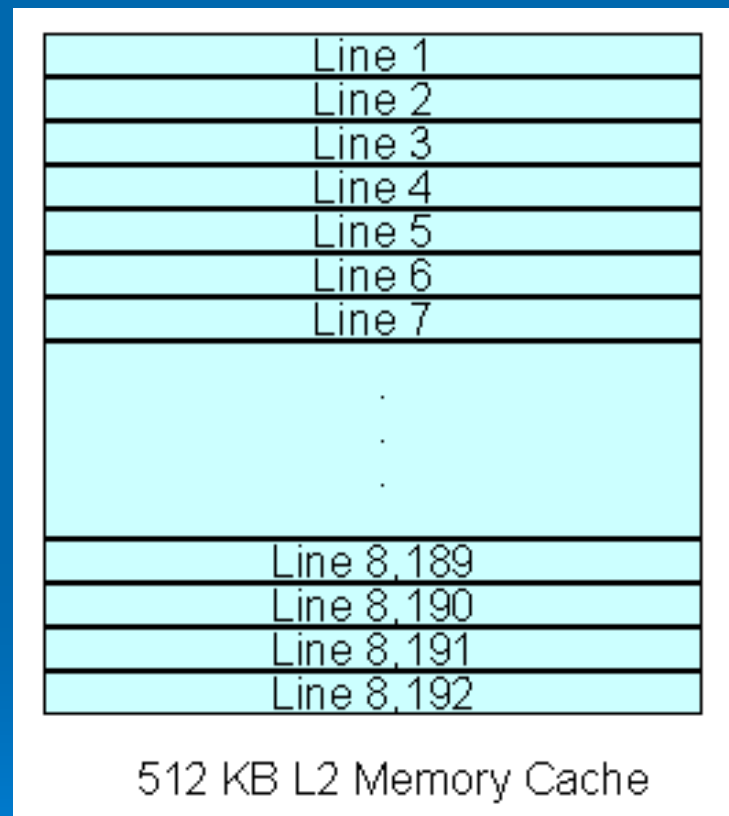
- The Level 1 and level 2 Data Cache
- The cache lines are each 64 Bytes.
- This gives $8192 \text{ bytes} / 64 \text{ bytes} = 128$ cache lines.

➤ How Does The Cache Manage the Cache Lines?

- ❑ **Associativity** describes how the data is stored in the cache.
- Direct Mapped (Associativity = 1) means each line has its own slot. (Analogy: Each person gets their own mailbox.)
- X-way Associativity means X cache lines share a slot. (All the “A’s” share a mailbox, but it’s a bigger mailbox.)
- Fully associative means all cache lines share the same possible places. (All the letters are put into one giant mailbox.)

Cache Memory

- Divided into lines
- Each lines holding 16 to 128 byte lines depending on the CPU
- On the majority of current CPUs the memory cache is organized in 64-byte lines

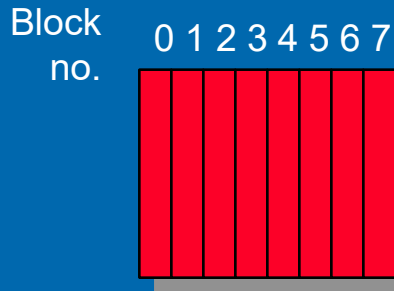


Where can a block be **placed** in the Cache?

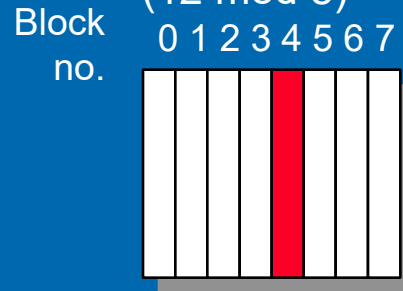
➤ Block 12 placed in 8 block cache:

- Fully associative, direct mapped, 2-way set associative
- S.A. Mapping = Block Number Modulo Number Sets

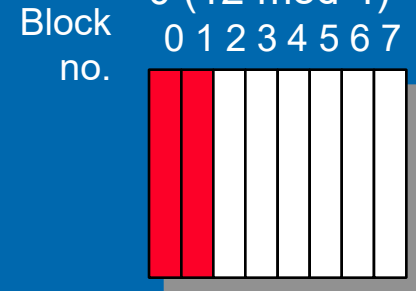
Fully associative:
block 12 can go
anywhere



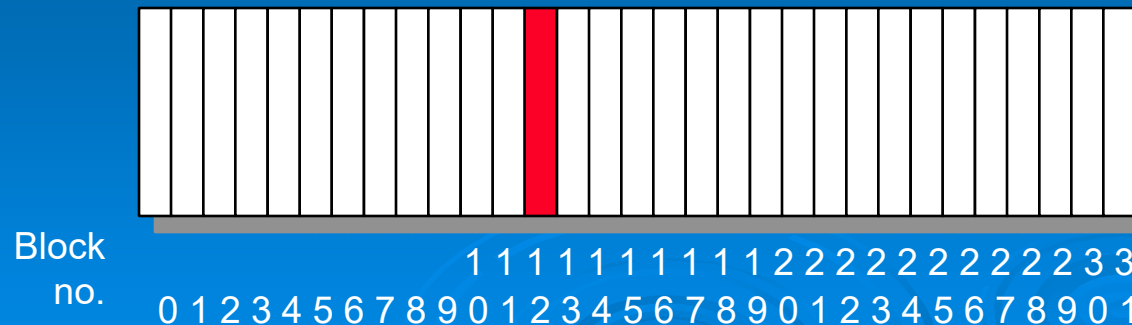
Direct mapped:
block 12 can go
only into block 4
(12 mod 8)



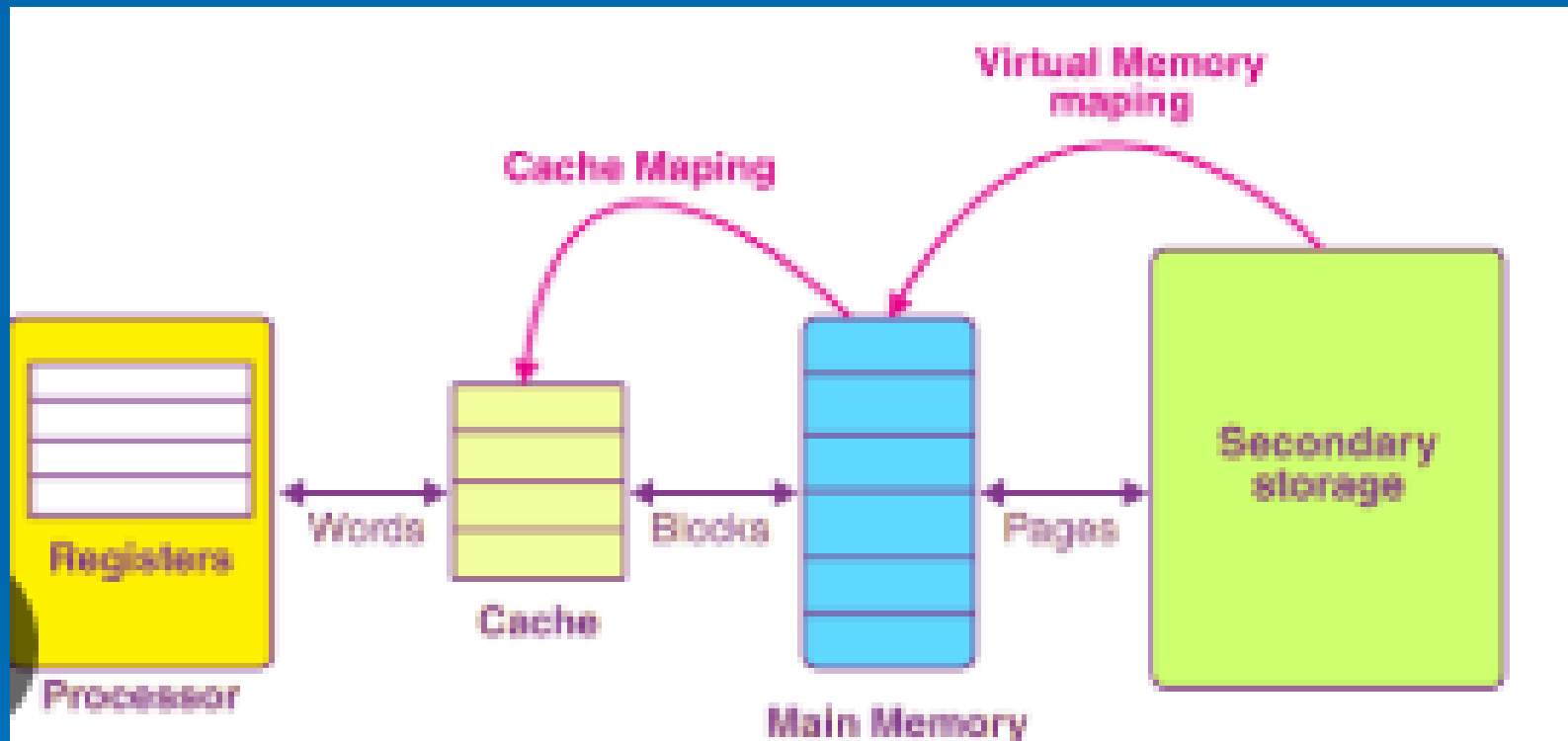
Set associative:
block 12 can go
anywhere in set
0 (12 mod 4)



Block-frame address



Mapping Function



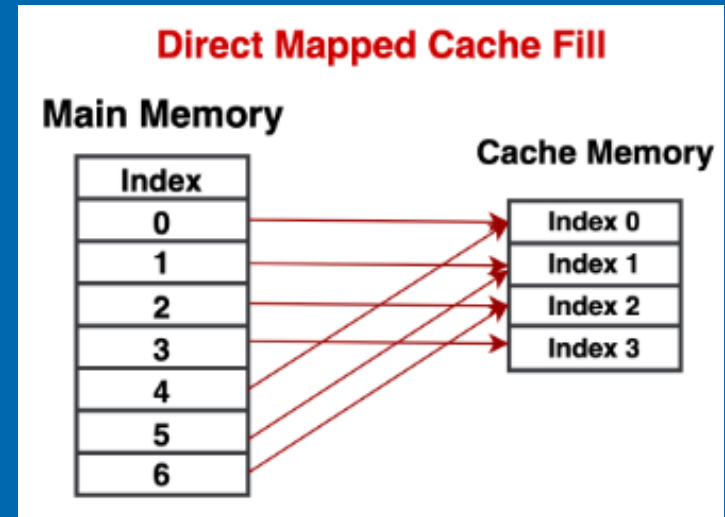
Cache mapping refers to a technique using which the content present in the main memory is brought into the memory of the cache.

Cache Mapping Techniques

- Cache mapping is the method by which the contents of main memory are brought into the cache and referenced by the CPU
- The mapping method used directly affects the performance of the entire computer system.

Direct Mapping

- Main memory locations can only be copied into one location in the cache.
- Direct mapping is accomplished by dividing a main memory address into a tag and an index field, where the index identifies a single, specific cache line for a memory block.



Example content of a Direct-Mapped Cache

- Example of the **content** of an **Direct-Mapped cache**:

Direct Mapped Cache

Valid Flag	Block Number	Value
0	1	1232
1	1	12
2	1	98
3	0	
4	1	0
5	1	0
6	1	2
7	1	1

0	14	0
1	17	1
2	98	2
3	162	3
4	8172	4
5	1	5
6	283	6
7	1283	7
8	1232	0
9	123	1
10	98	2
11	123	3
12	12	4
13	19982	5
14	1	6
15	993	7
16	982	0
17	12	1
18	1123	2
19	11	3
20	121	4
21	11	5
22	1232	6
23	1	7

Direct Mapping

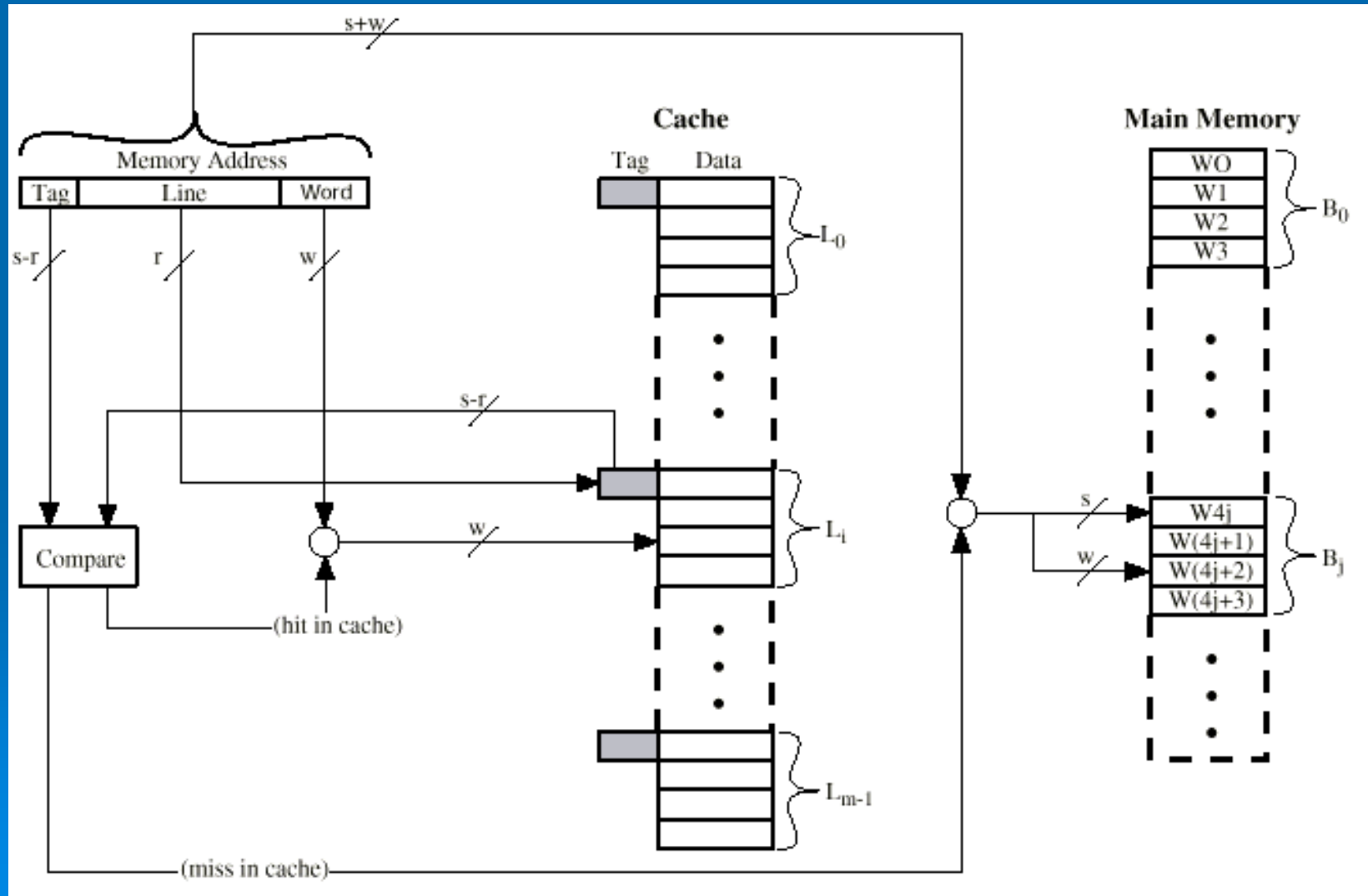
- Each block of main memory maps to only one cache line
 - i.e. if a block is in cache, it must be in one specific place
- Address is in two parts
- Least Significant w bits identify unique word
- Most Significant s bits specify one memory block
- The MSBs are split into a cache line field r and a tag of $s-r$ (most significant)

Direct Mapping Address Structure

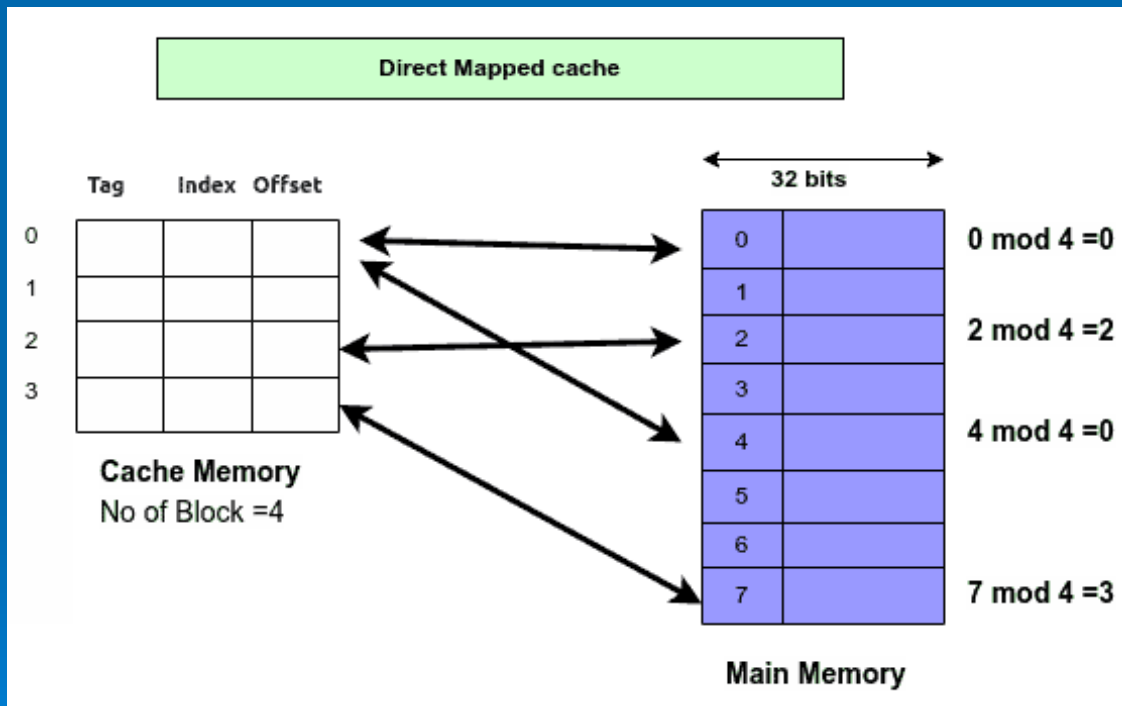
Tag s-r	Line or Slot r	Word w
8	14	2

- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
 - 8 bit tag (=22-14)
 - 14 bit slot or line
- No two blocks in the same line have the same Tag field
- Check contents of cache by finding line and checking Tag

Direct Mapping Cache Organization



Direct Mapping Cache Organization: Example



$$i = j \text{ modulo } m$$

where

i = cache line number

j = main memory block number

m = number of lines in the cache

Associative Mapping

- A main memory block can load into any line of cache
- Memory address is interpreted as tag and word
- Tag uniquely identifies block of memory
- Every line's tag is examined for a match
- Cache searching gets expensive

Associative Mapping Address Structure

Tag 22 bit	Word 2 bit
------------	---------------

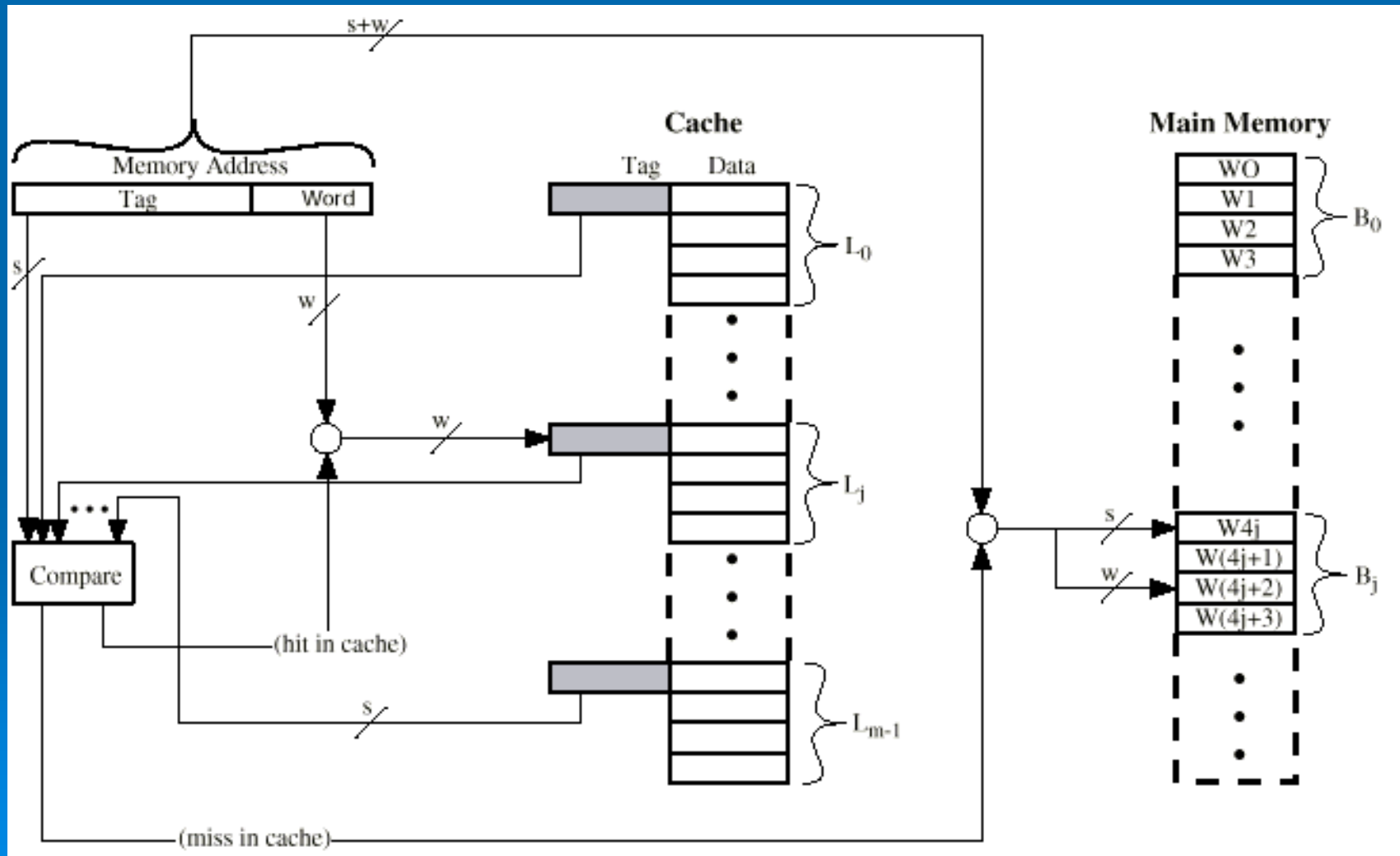
- 22 bit tag stored with each 32 bit block of data
- Compare tag field with tag entry in cache to check for hit
- Least significant 2 bits of address identify which 16 bit word is required from 32 bit data block
- e.g.

Address	Tag	Data	Cache line
• FFFFFC	FFFFFC	24682468	3FFF

Full Associative Mapping

- the most complex
- but it is most flexible with regards to where data can reside
- A newly read block of main memory can be placed anywhere in the full associative cache
- If the cache is full, a replacement algorithm is used to determine which block in the cache gets replaced by the new data
- it needs to keep track of what memory locations are loaded inside the memory cache

Fully Associative Cache Organization



Set Associative Mapping

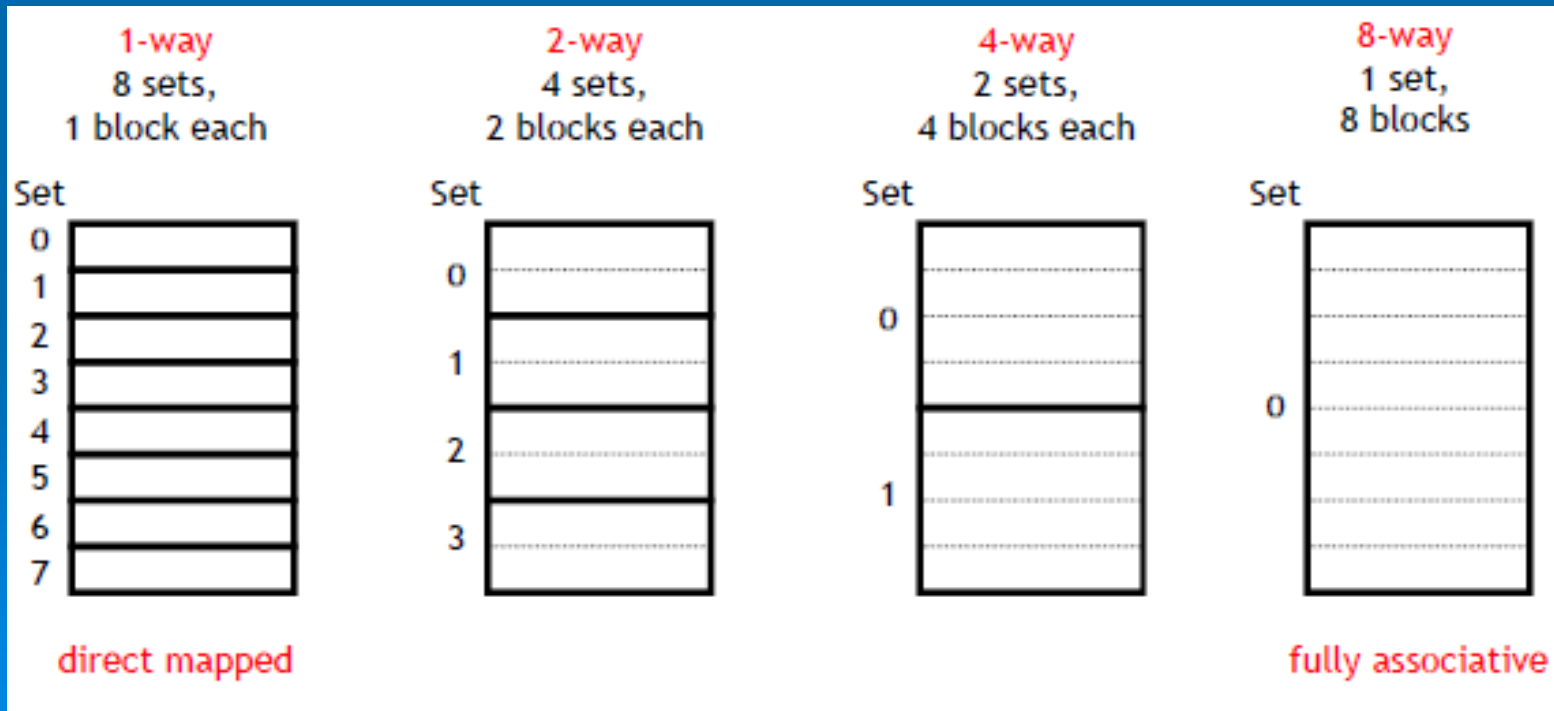
- combines the best of direct and associative cache mapping techniques
- As with a direct mapped cache, blocks of main memory data will still map into as specific set, but they can now be in any N-cache block frames within each set

Set Associative Mapping

- Cache is divided into group of blocks called sets
- Each memory address maps to exactly one set in the cache, but data may be placed in any block within that set.
- If each set has 2^x blocks, the cache is an 2^x -way associative cache.

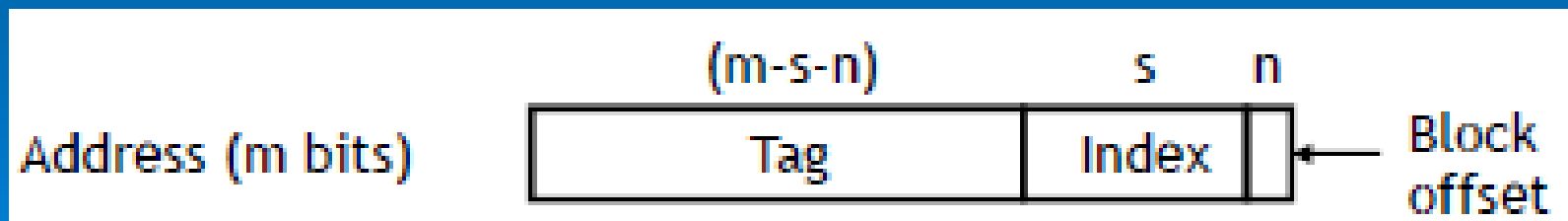
Set Associative Possible Organizations

Here are several possible organizations of an eight-block cache.



Locating a Set Associative Block

- We can determine where a memory address belongs in an associative cache in a similar way as before.
- If a cache has 2^s sets and each block has 2^n bytes, the memory address can be partitioned as follows.



Locating a Set Associative Block

- Our arithmetic computations now compute a set index, to select a set within the cache instead of an individual block.

Block Offset = Memory Address mod 2^n

Block Address = Memory Address / 2^n

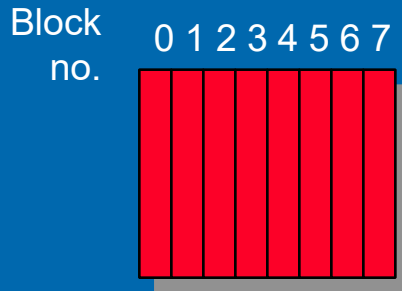
Set Index = Block Address mod 2^s

Where can a block be placed in the Cache?

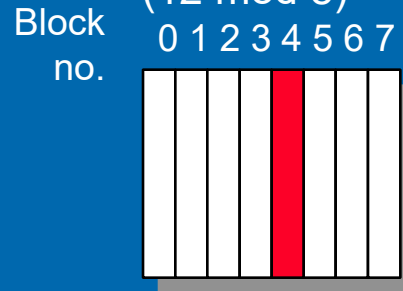
➤ Block 12 placed in 8 block cache:

- Fully associative, direct mapped, 2-way set associative
- S.A. Mapping = Block Number Modulo Number Sets

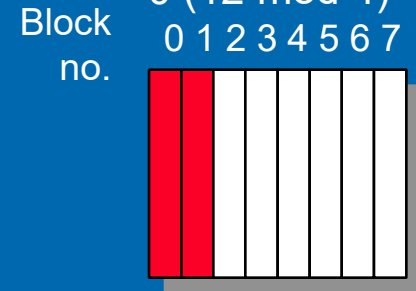
Fully associative:
block 12 can go
anywhere



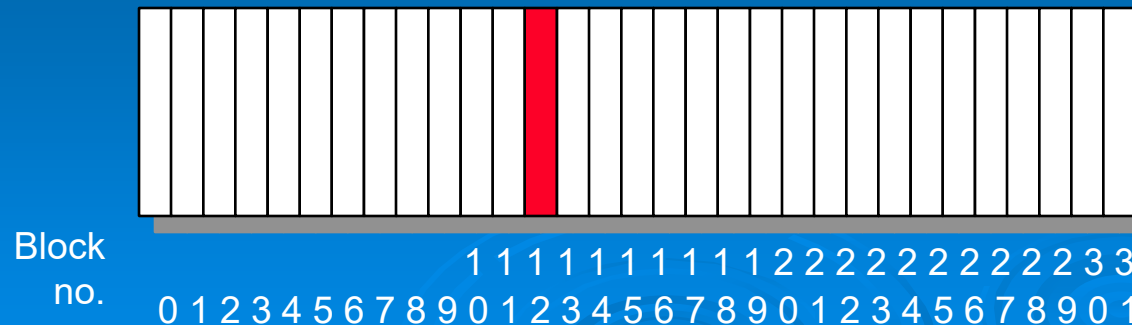
Direct mapped:
block 12 can go
only into block 4
(12 mod 8)



Set associative:
block 12 can go
anywhere in set
0 (12 mod 4)



Block-frame address



Replacement Algorithm

- Replacement algorithms/policies are used to attain optimized usage of cache.
- When cache is full, then replacement policies decide which piece of data is replaced to make space for new data that is currently being used.

Replacement Algorithms for Direct mapping

- No choice
- Each block only maps to one line
- Replace that line

Replacement Algorithms for Associative

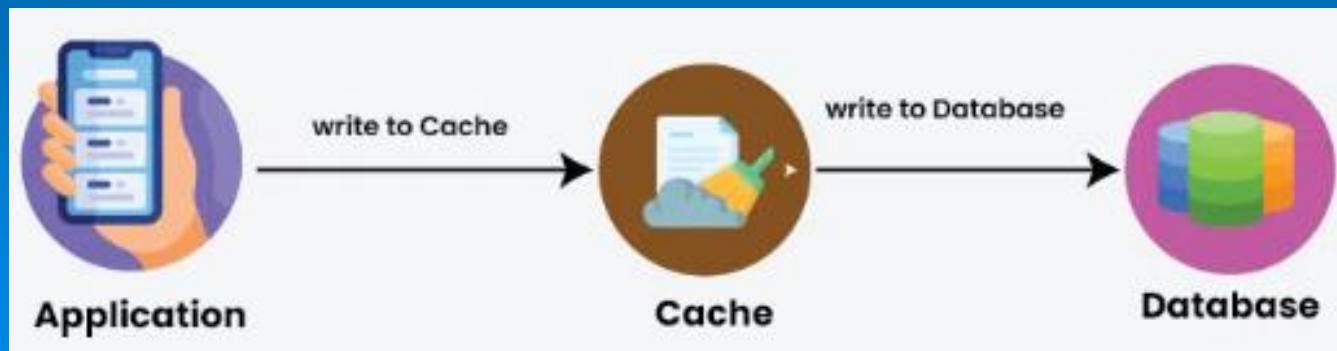
- Least Recently used (LRU)- replace the cache line that has been in the cache the longest with no references to it
 - First in first out (FIFO) - replace block that has been in cache longest
 - Least frequently used - replace block which has had fewest hits
 - Random - pick a line at random from the candidate lines when a cache miss occurs.
- 
- The bottom right corner of the slide features a decorative graphic consisting of several sets of concentric circles, resembling ripples in water, rendered in a lighter blue shade against the main blue background.

Write Policy

- Cache write policies dictate how data modifications in the cache are propagated to the main memory.
- The primary goal of these policies is to balance performance and data consistency.
- Write policies determine when and how the changes made to the cached data are written back to the main memory

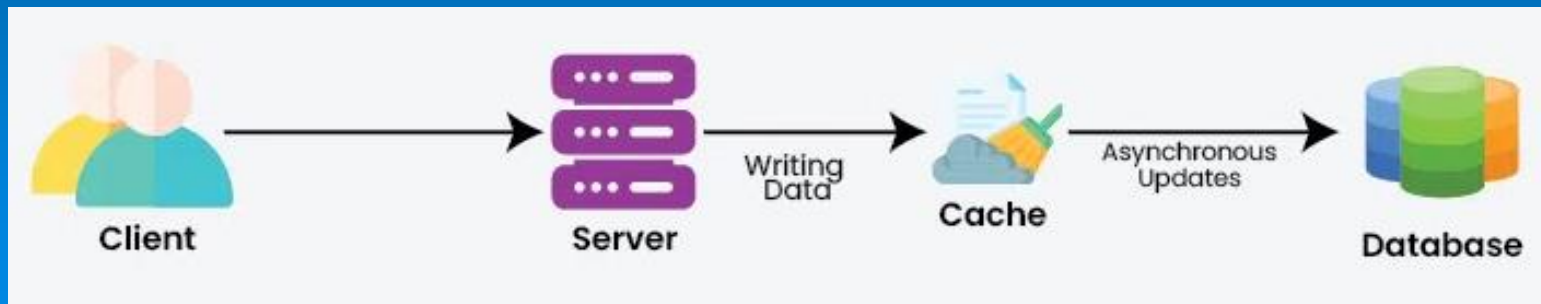
Write through

- All writes go to main memory as well as cache
- Multiple CPUs can monitor main memory traffic to keep local (to CPU) cache up to date
- Disadvantages
 - Slower write operations due to simultaneous writes.
 - Increased memory bandwidth usage.



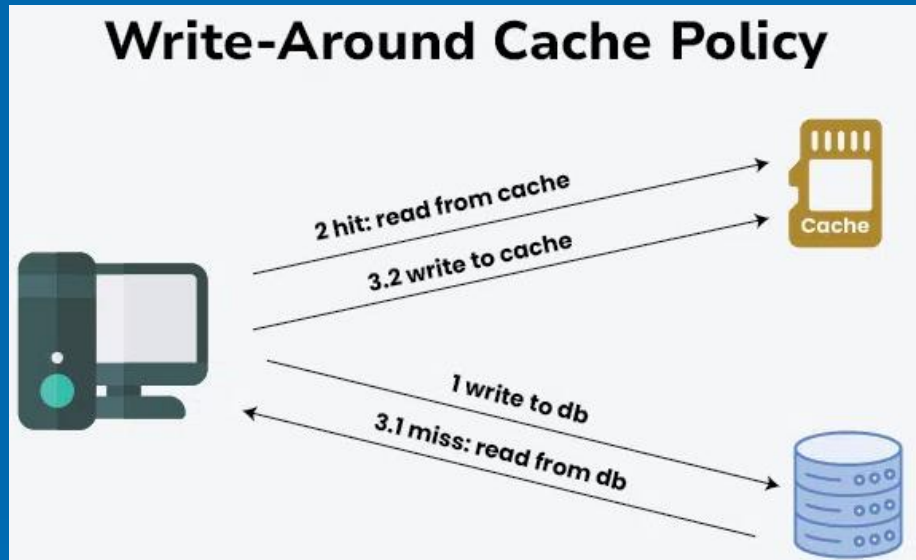
Write back

- Updates initially made in cache only
- Update bit for cache slot is set when update occurs
- If block is to be replaced, write to main memory only if update bit is set
- Advantages
 - Faster write operations as writes are initially only to the cache.
 - Reduced memory bandwidth usage.
- Disadvantages
 - Requires mechanisms to ensure data consistency.
 - Complexity in handling cache coherency



Write-Around

- Data is written directly to the main memory, bypassing the cache. The cache is only updated if the same data is read again.
- Advantages
 - Reduces cache pollution with infrequently accessed data.
 - Suitable for workloads with low write locality.
- Disadvantages
 - May lead to slower write operations.
 - Potential for cache misses on subsequent reads.



Virtual Memory

- Divides physical memory into blocks and allocate them to different processes.
- VM's purpose is to enlarge the set of memory addresses a program can utilize

Virtual Memory

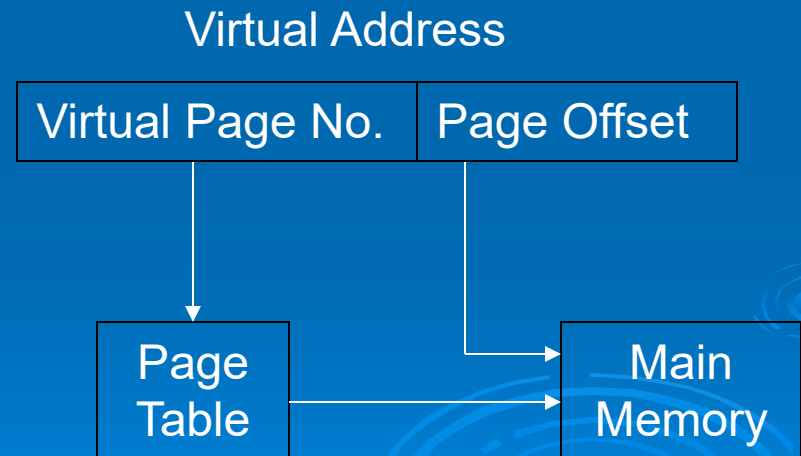
- With virtual memory, what the computer can do is look at RAM for areas that have not been used recently and copy them onto the hard disk. This frees up space in RAM to load the new application.
- The area of the hard disk that stores the RAM image is called a **page file** that holds **pages** of RAM on the hard disk

Virtual Memory

- Page is a fixed sized memory addresses
- The area of the hard disk that stores the RAM image is called a **page file** that holds **pages** of RAM on the hard disk

Page ID: Address Translation

- OS performs address translation using page table
 - Each process has its own page table
 - OS knows address of each process's page table
 - Page table is an array of Page Table Entries
 - One entry for each VPN of each process, indexed by VPN
 - Each PTE contains
 - Phys. Page Number
 - Permissions
 - Dirty bit
 - LRU
 - ~4bytes total



References

- William Stallings. Computer Architecture and Organization, 11th Edition 2019.
- Structured Computer Organization by Andrew Tannenbaum
- Vasanth Venkatachalam and Michael Franz. Power Reduction Techniques For Microprocessor Systems. ACM Computing Surveys. 2005. pp 195-233.
- Prof. David Brooks. Computer Architecture. Lecture 17: Virtual Memory. Harvard University. 2005